

Algorithms

Chapter 12 Binary Search Trees

Associate Professor: Ching-Chi Lin

林清池 副教授

chingchi.lin@gmail.com

Department of Computer Science and Engineering
National Taiwan Ocean University

Outline

- ▶ **What is a binary search tree?**
- ▶ Querying a binary search tree
- ▶ Insertion and deletion

Overview

Search trees: {
a. search
b. min, max
c. predecessor, successor (前一个 or 下一个)
d. insert, delete

- ▶ **Search trees** are data structures that support many dynamic-set operations.

- ▶ Dynamic-set operations includes SEARCH, MINIMUM, MAXIMUM, PREDECESSOR, SUCCESSOR, INSERT, and DELETE.

- ▶ Can be used as both a dictionary and as a priority queue. (a, d) (b, d)

- ▶ Basic operations take time proportional to the height of the tree, i.e., $\Theta(h)$. 基本動作時間: $\Theta(h) \rightarrow$ 樹高

- ▶ For complete binary tree with n nodes: worst case $\Theta(\lg n)$.

- ▶ For linear chain of n nodes: worst case $\Theta(n)$. 從 $\Theta(\lg n) \sim \Theta(n)$
(樹高可能從 $\lg n \sim n$, 可能歪斜)

- ▶ Different types of search trees include **binary search trees**, **red-black trees** (Chapter 13), and **B-trees** (Chapter 18).

可控制高度

Binary search trees

- ▶ We represent a binary tree by a linked data structure in which each node is an object.
 - ▶ Each node contains the fields
 - ▶ *key* and possibly other satellite data.
 - ▶ *left*: points to left child.
 - ▶ *right*: points to right child.
 - ▶ *p*: points to parent. $p[\text{root}[T]] = \text{NIL}$.
 - ▶ Stored keys must satisfy the **binary-search-tree property**.
 - ▶ If y is in left subtree of x , then $\text{key}[y] < \text{key}[x]$. 左子樹 < 我
 - ▶ If y is in right subtree of x , then $\text{key}[y] \geq \text{key}[x]$. 右子樹 $\geq$ 我
- 左邊小、右邊大

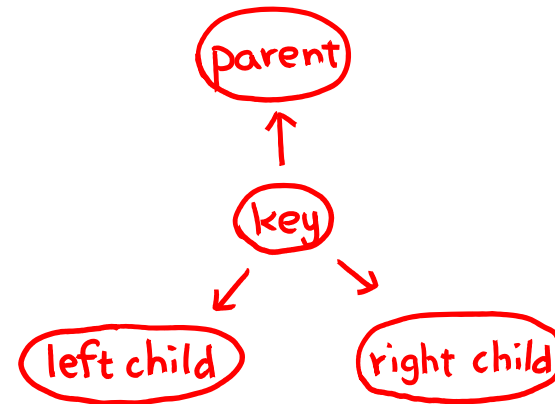
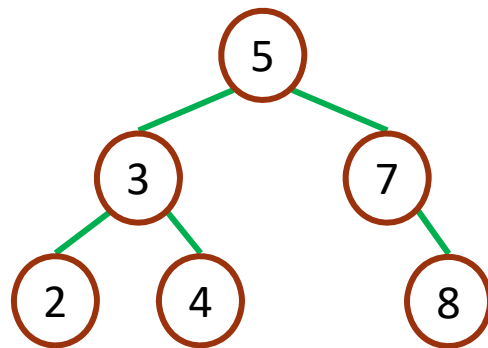
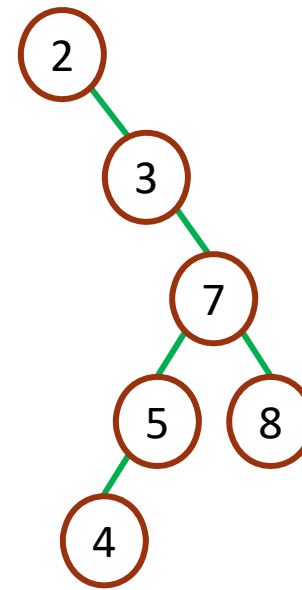


Figure 12.1 Binary search trees

(a) (b) 擁有相同性質，
但 (a) 樹高較低，
較有效率



(a)



(b)

- ▶ A binary search tree on 6 nodes with height 2.
- ▶ A less efficient binary search tree with height 4 that contains the same keys.

Inorder tree walk

有子 binary search tree 的性質，
我們可將 key 值由小到大印出

- ▶ The binary-search-tree property allows us to print keys in a binary search tree in order, recursively.
- ▶ Elements are printed in monotonically increasing order.

INORDER-TREE-WALK(x)

1. **if** $x \neq \text{NIL}$
2. INORDER-TREE-WALK($\text{left}[x]$) 印左子樹
3. print $\text{key}[x]$ 印自己
4. INORDER-TREE-WALK($\text{right}[x]$) 印右子樹

- ▶ The inorder tree walk prints the keys in each of the two binary search trees from Figure 12.1 in the order 2, 3, 4, 5, 7, 8.

Properties of binary search trees

- ▶ **Theorem** If x is the root of an n -node subtree, then the call $\text{INORDER-TREE-WALK}(x)$ takes $\Theta(n)$.

Proof: $\text{inorder_tree_walk}(x)$ 執行時間為 $\Theta(n)$

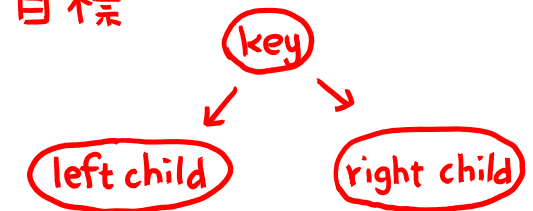
- ▶ $T(0) = c$, as It takes constant time on an empty subtree.
- ▶ Left subtree has k nodes and right subtree has $n-k-1$ nodes.
- ▶ d : the time to execute $\text{INORDER-TREE-WALK}(x)$, exclusive of the time spent in recursive calls. 假設左樹有 k 個 node, 右樹有 $n-k-1$ 個 node
- ▶ Prove by substitution method: $T(n) = (c+d)n + c$. ← 目標
- ▶ For $n = 0$, we have $(c+d) \cdot 0 + c = c = T(0)$.

- ▶ For $n > 0$, $T(n) = T(k) + T(n-k-1) + d$

$$= ((c+d)k + c) + ((c+d)(n-k-1) + c) + d$$

$$= (c+d)n + c - (c+d) + c + d$$

$$= (c+d)n + c.$$



只執行第1行 (if $x \neq \text{NIL}$)

c : 當樹為空所需時間 (node 為空)

d : 只執行 1, 3 行時間 (node 不為空)

(不斷不斷不斷不斷)

7

$T(n)$ = 即左子樹所需時間 + 即右子樹所需時間 + 執行 1, 3 行時間

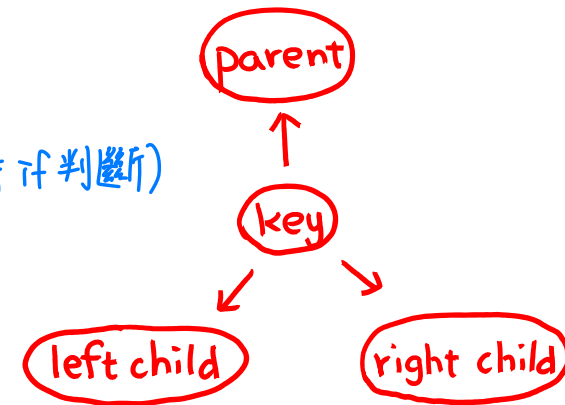
Outline

- ▶ What is a binary search tree?
- ▶ **Querying a binary search tree**
- ▶ Insertion and deletion

c: 當樹為空所需時間 (node是空的、只執行第一行if判斷)

d: 只執行1, 3行時間 (node不空、判斷+印自己)

$$T(n) = dn + c(n+1) = (c+d)n + c$$



節點 $\Rightarrow$ 有 n 個

子節點指標為空 $\Rightarrow$ 共有 $n+1$ 個

I. 每個節點都有父節點, root 沒有 $\Rightarrow$ 有 $n-1$ 個子節點

II. 每個節點配置 2 個 $\Rightarrow n * (\text{左子} + \text{右子}) = 2n$

III. 為空 = 全 - 用到 = $2n - (n-1) = n+1$

Operations on binary search trees

- ▶ We shall examine SEARCH, MINIMUM, MAXIMUM, SUCCESSOR, and PREDECESSOR operations. 時間 $\leq$ 樹高的常數倍

- ▶ The running times of these operations are all $O(h)$.

TREE-SEARCH(x, k) 遞迴版

1. **if** $x = \text{NIL}$ or $k = \text{key}[x]$
2. **then return** x
3. **if** $k < \text{key}[x]$
4. **then return** TREE-SEARCH($\text{left}[x], k$)
5. **else return** TREE-SEARCH($\text{right}[x], k$)

非遞迴 $\rightarrow$ 迴圈

ITERATIVE-TREE-SEARCH(x, k)

1. **while** $x \neq \text{NIL}$ and $k \neq \text{key}[x]$
2. **if** $k < \text{key}[x]$
3. **then** $x \leftarrow \text{left}[x]$
4. **else** $x \leftarrow \text{right}[x]$
5. **return** x

雖然都是 $\leq$ 樹高常數倍, 但非遞迴較快

- ▶ On most computers, the iterative version is more efficient.
遞迴時要將目前的資料存到 stack, 返回時要 pop (compiler 做的)
- ▶ **Time:** The algorithm visiting nodes on a downward path from the root. Thus, running time is $O(h)$.

Minimum and maximum

- ▶ The binary-search-tree property guarantees that
 - ▶ the minimum key of a binary search tree is located at the leftmost node, and 最小：最左的葉
 - ▶ the maximum key of a binary search tree is located at the rightmost node. 最大：最右的葉
- ▶ Traverse the appropriate pointers (*left* or *right*) until NIL is reached.

TREE-MINIMUM(x)

1. **while** $left[x] \neq NIL$
2. **do** $x \leftarrow left[x]$
3. **return** x

TREE-MAXIMUM(x)

1. **while** $right[x] \neq NIL$
2. **do** $x \leftarrow right[x]$
3. **return** x

- ▶ **Time:** Both procedures visit nodes that form a downward path from the root to a leaf. Both procedures run in $O(h)$ time.
-

Successor and predecessor_{1/2}

- ▶ Assuming that all keys are distinct, the successor of a node x is the node y such that $key[y]$ is the smallest key $> key[x]$.
- ▶ The structure of a binary search tree allows us to determine the successor of a node without ever comparing keys.
- ▶ If x has the largest key in the binary search tree, then we say that x 's successor is NIL.
- ▶ There are two cases:
 - ▶ If node x has a non-empty right subtree, then x 's successor is the minimum in x 's right subtree.
 - ▶ If node x has an empty right subtree and x has a successor y , then y is the lowest ancestor of x whose left child is also an ancestor of x .

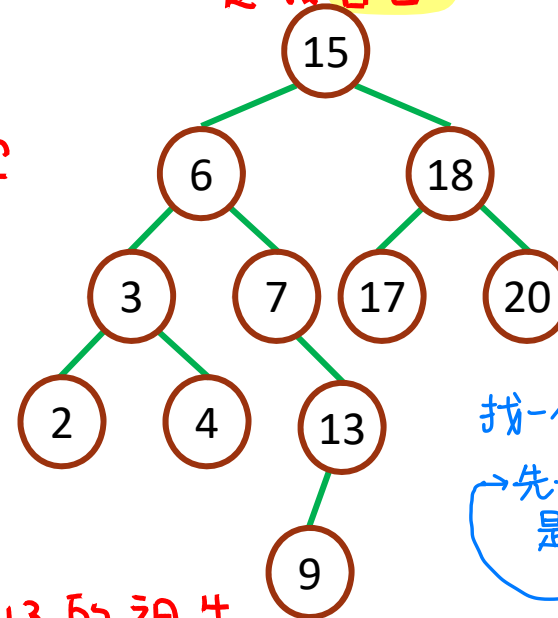
Successor and predecessor_{2/2}

右子樹 < 不空: 右子樹是最小的那個
空: 最小祖先滿足其左子
I. 也是祖先 or
II. 是我自己 是自己的祖先

TREE-SUCCESSOR(x)

1. **if** $right[x] \neq \text{NIL}$
2. **return** TREE-MINIMUM($right[x]$)
3. $y \leftarrow p[x]$
4. **while** $y \neq \text{NIL}$ and $x = right[y]$
5. $x \leftarrow y$
6. $y \leftarrow p[y]$
7. **return** y

空



找一個人的 Successor
→ 先去問他爸
是不是左子 or 祖先

13 的左子是 9 自己 15 的左子是 6, 6 也是 13 的祖先

- ▶ The successor of the node with key 13 is the node with key 15.
- ▶ **Time:** Since we either follow a path up the tree or follow a path down the tree. The running time is $O(h)$.
- ▶ TREE-PREDECESSOR is symmetric to TREE-SUCCESSOR.

Outline

- ▶ What is a binary search tree?
- ▶ Querying a binary search tree
- ▶ **Insertion and deletion**

Insertion and deletion *insert 和 delete 会改变 樹*

- ▶ The operations of insertion and deletion cause the dynamic set represented by a binary search tree to change.
- ▶ The binary-search-tree property must hold after the change.
- ▶ Insertion is more straightforward than deletion.

insert 比較容易

*在做完 insert 和 delete 後, binary-search-tree
的性質要維持下去*

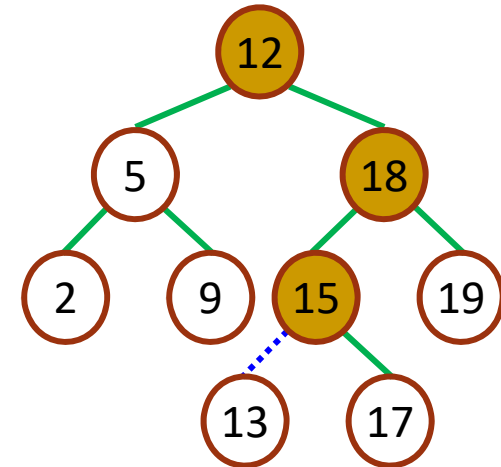
Insertion

TREE-INSERT(T, z)

1. $y \leftarrow \text{NIL}; x \leftarrow \text{root}[T]$
2. **while** $x \neq \text{NIL}$
3. $y \leftarrow x$
4. **if** $\text{key}[z] < \text{key}[x]$
5. $x \leftarrow \text{left}[x]$
6. **else** $x \leftarrow \text{right}[x]$
7. $p[z] \leftarrow y$
8. **if** $y = \text{NIL}$
9. $\text{root}[T] \leftarrow z$ /* Tree T was empty */
10. **else if** $\text{key}[z] < \text{key}[y]$
11. $\text{left}[y] \leftarrow z$
12. **else** $\text{right}[y] \leftarrow z$

找父親 y

比父親大放右
比父親小放左



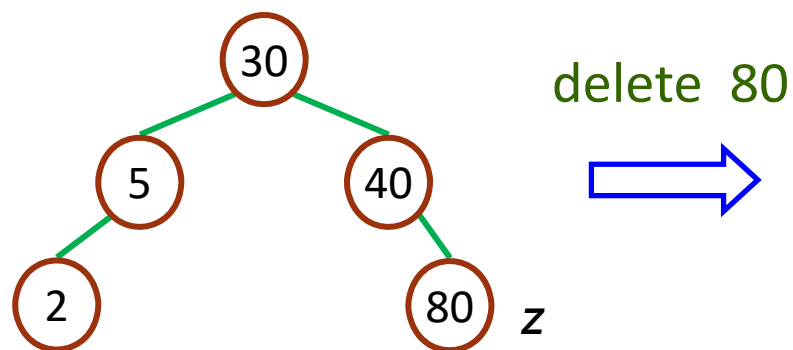
Inserting an item
with key 13

- **Time:** Since we follow a path down the tree.
The running time is $O(h)$.

Deletion

- ▶ TREE-DELETE is broken into three cases.
 - ▶ **Case 1:** z has no children. 沒有兒子, 直接刪
 - ▶ Delete z by making the parent of z point to NIL, instead of to z.
 - ▶ **Case 2:** z has one child. 有一個兒子, 將兒子連到父親
 - ▶ Delete z by making the parent of z point to z's child, instead of to z.
 - ▶ **Case 3:** z has two children.
 - ▶ z's successor y has either no children or one child.
(y is the minimum node with no left child in z's right subtree.)
 - ▶ Delete y from the tree (via Case 1 or 2).
 - ▶ Replace z's key and satellite data with y's.
- 可以證明下一個最多只有一個兒子
→ 最多 delete 2 次
- ① 找下一個 Successor
② 將下一個 copy 過來
③ 將下一個 刪除

Case 1:



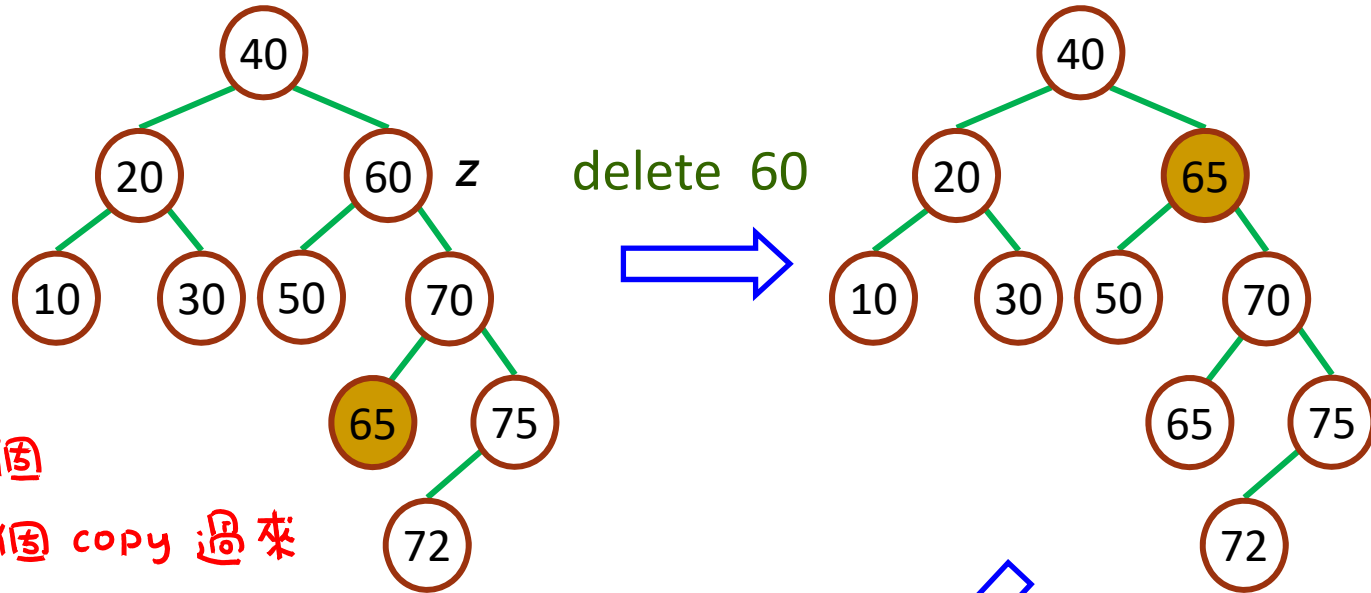
沒有兒子, 直接刪

Case 2:

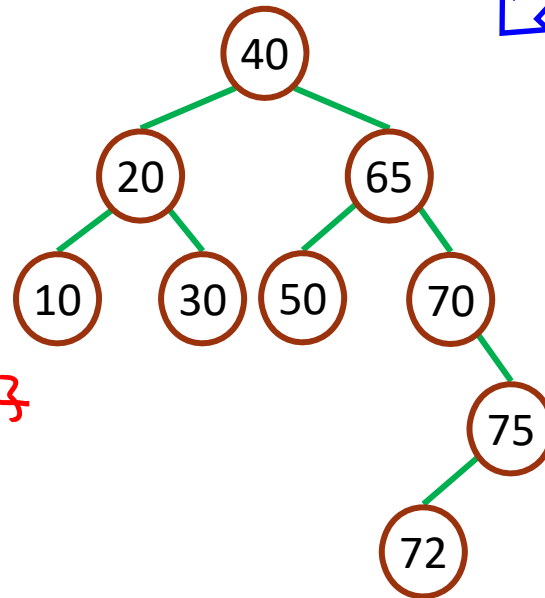


有一個兒子, 將兒子連到父親

Case 3:



- ① 找下一個
- ② 將下一個 copy 過來
- ③ 將下一個刪除



(Successor 是右子樹中的最小、最多只有右子樹)
可以證明下一個最多只有一個兒子
→ 沒有左子
→ 最多 delete 2 次

Deletion

TREE-DELETE(T, z)

1. **if** $left[z] = \text{NIL}$ or $right[z] = \text{NIL}$
 2. $y \leftarrow z$
 3. **else** $y \leftarrow \text{TREE-SUCCESSOR}(z)$
 4. **if** $left[y] \neq \text{NIL}$
 5. $x \leftarrow left[y]$
 6. **else** $x \leftarrow right[y]$
 7. **if** $x \neq \text{NIL}$
 8. $p[x] \leftarrow p[y]$
 9. **if** $p[y] = \text{NIL}$
 10. $root[T] \leftarrow x$
 11. **else if** $y = left[p[y]]$
 12. $left[p[y]] \leftarrow x$
 13. **else** $right[p[y]] \leftarrow x$
 14. **if** $y \neq z$
 15. $key[z] \leftarrow key[y]$
 16. copy y 's satellite data into z
 17. **return** y
- Time: $O(h)$.

y : 真正刪除的節點

x : y 唯一 - 有可能的兒子

設定 x 的新父親

設定 x 為新父親的左子或右子

y 和 z 不同節點: case 3

→ 複製 y 的資料到 z